

SISTEMAS OPERACIONAIS

VI - Scheduling da CPU

Professor: PhD. Eduardo Duarte

Agenda

01 Escalonador da CPU

02 Critérios de Escalonamento

03 Algoritmos de Escalonamento

Parte I - Conceitos sobre Scheduling da CPU

Scheduling da CPU

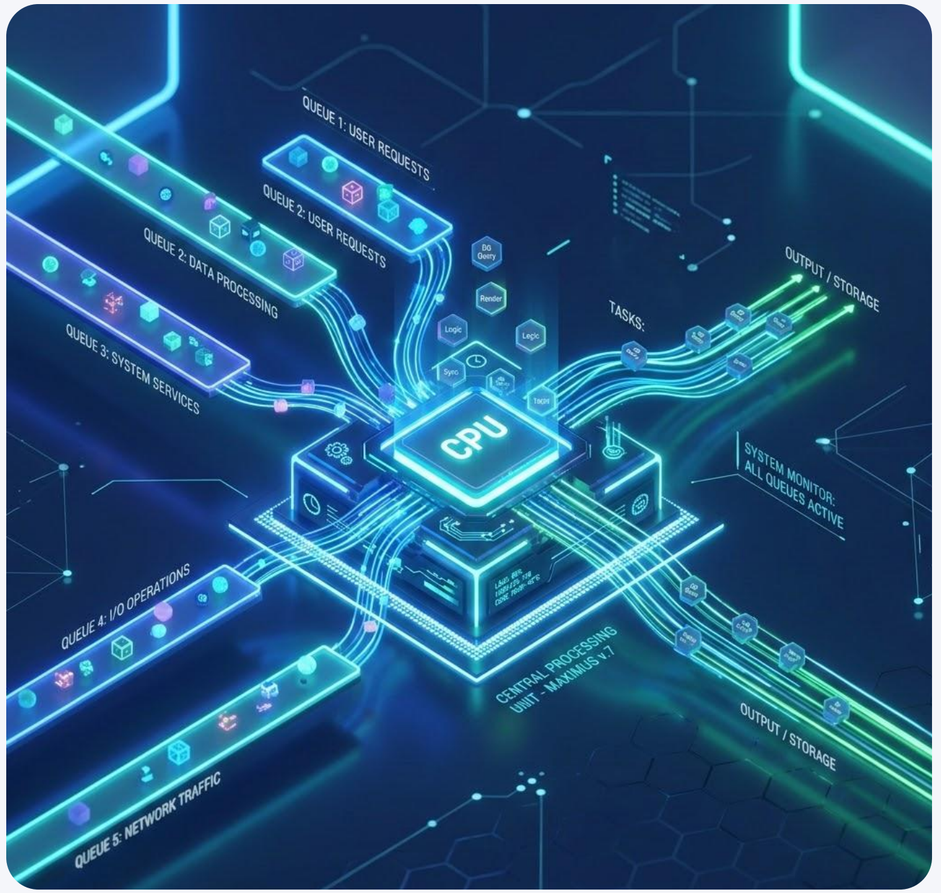
A base fundamental dos sistemas operacionais multiprogramados

1. Alternância de Processos

O Sistema Operacional alterna a CPU entre os diferentes processos para otimizar o fluxo de trabalho e manter o computador em constante produção.

2. Objetivo da Multiprogramação

Garantir que sempre haja algum processo em execução, maximizando o uso da CPU e eliminando tempos ociosos.



Scheduling da CPU

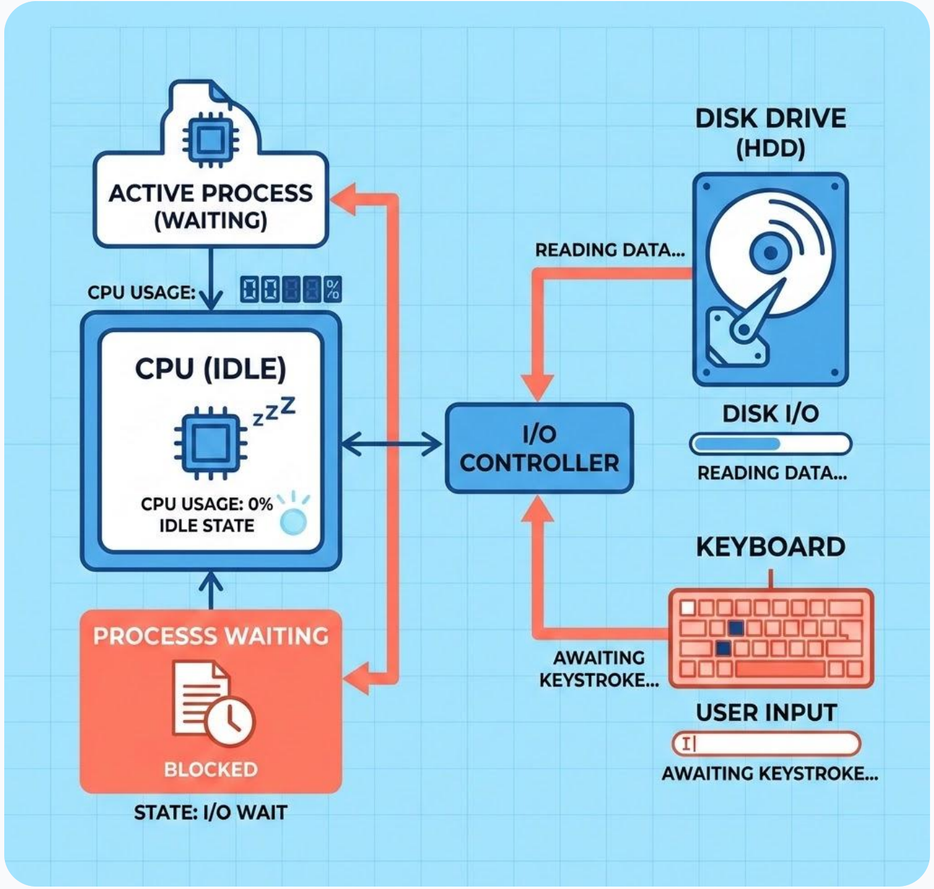
A ideia é relativamente simples

1. Execução e Solicitação de I/O

Um **processo é executado** continuamente até necessitar aguardar a resposta de uma **solicitação de I/O** (ex: leitura de disco, rede).

2. O Problema da Ociosidade

Em sistemas simples (sem multiprogramação), a **CPU permanece totalmente ociosa** durante esse tempo de espera **(Tempo desperdiçado)**.



Scheduling da CPU

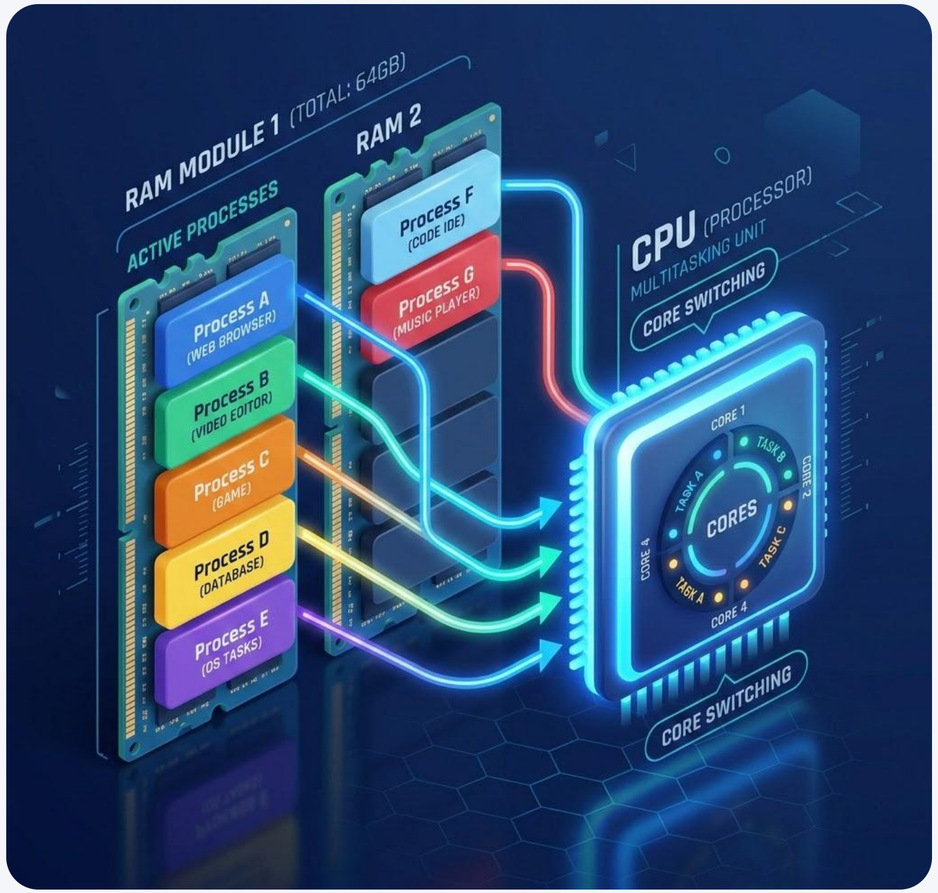
O papel da multiprogramação no uso produtivo do processador

1. Multiprogramação e Memória

Vários processos são mantidos na memória ao mesmo tempo, permitindo usar o tempo do processador de forma altamente produtiva.

2. Realocação Dinâmica da CPU

Quando um **processo precisa esperar**, o Sistema Operacional desvincula a CPU e a designa imediatamente a outro processo.



Ciclos de Picos de CPU-I/O

A alternância contínua entre processamento ativo e espera por entrada/saída

1. Propriedade do Scheduling

O sucesso do escalonamento depende de uma característica dos processos: a **sequência intercalada** de execuções de CPU e esperas de I/O.

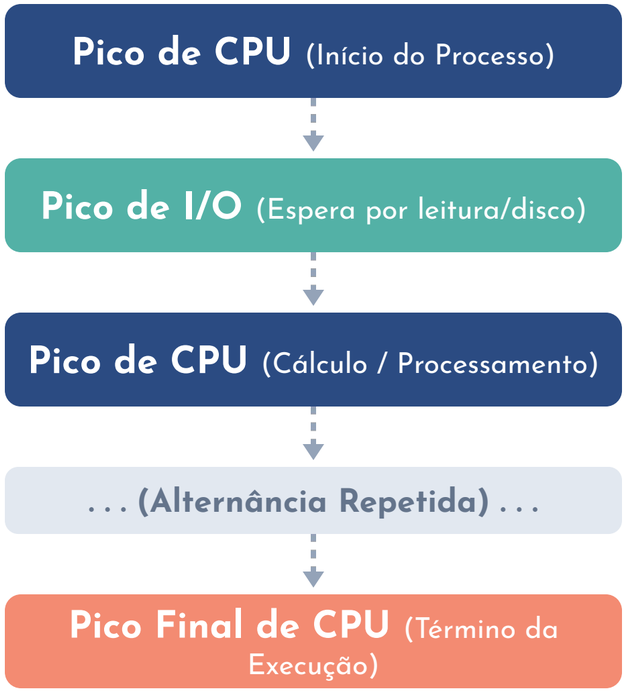
2. Alternância de Estados

Os processos variam constantemente entre esses dois estados. A CPU executa até precisar de uma operação de entrada/saída (I/O).

3. Início e Fim do Ciclo

A execução **inicia sempre com um pico de CPU**, alterna com I/O, e o último pico de CPU solicita o **encerramento** do processo.

Fluxo de Execução do Processo



Scheduler da CPU

Seleção e alocação dinâmica de processos em execução

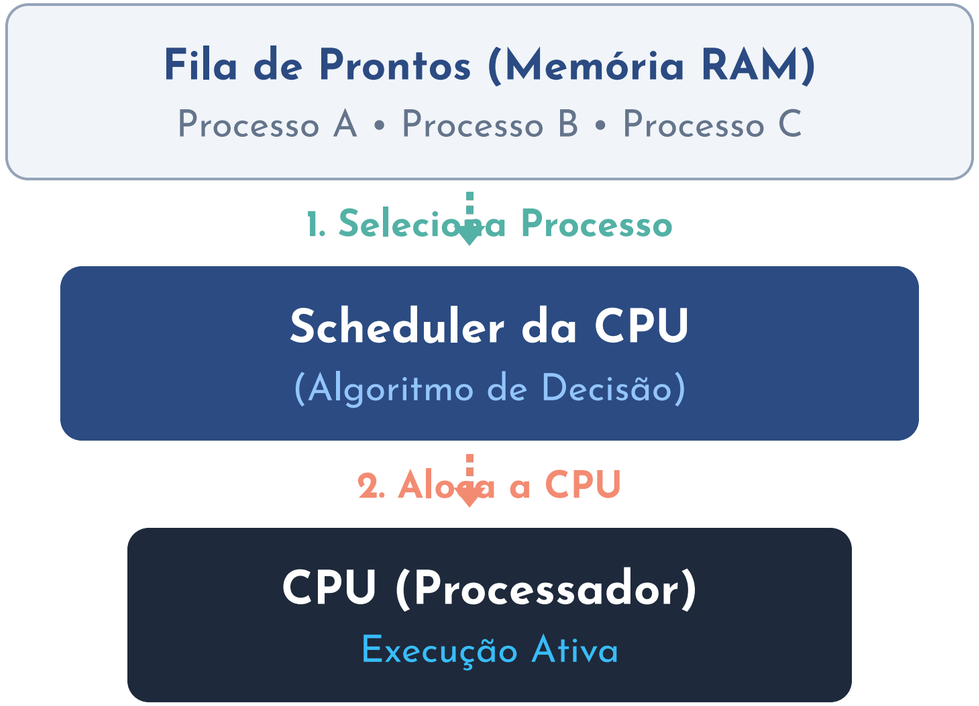
1. Quando a CPU fica ociosa

Sempre que o processador se torna ocioso, o sistema operacional deve selecionar um processo na **fila de prontos** para ser executado.

2. Quem faz a seleção?

O processo de seleção é realizado pelo **Scheduler de curto prazo** (ou **Scheduler da CPU**), que aloca o processador.

Fluxo de Escalonamento



Scheduler da CPU: Fila de Prontos

Formatos e estruturas de dados utilizadas para organizar os processos

A Fila de Prontos NÃO é necessariamente uma FIFO (First In - First Out)

Sua estrutura varia de acordo com o algoritmo e a estratégia de escalonamento do SO.

1. FIFO (Fila)

Ordem estrita de chegada. O primeiro a entrar é o primeiro a ser executado.

Visualização da Fila



Saída (CPU) <--- Entrada

2. Lista Encadeada

Nós conectados por ponteiros. Permite inserção e remoção flexíveis.

Nós Interligados

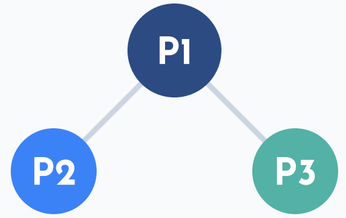


Acesso por Ponteiros (P_Head ... P_Tail)

3. Árvore / Heap

Organização hierárquica por prioridade (ex: Árvore Red-Black no Linux).

Hierarquia de Prioridades



Busca rápida do próximo processo

Scheduling Sem Preempção

Circunstâncias em que ocorrem as decisões de escalonamento da CPU

1. Mudança de Estado do Processo

O processo passa do estado de **Execução** para o estado de **Espera**.

Exemplos Práticos:

- Solicitação de E/S (I/O)
- Invocação de ***wait()*** aguardando processo-filho

Diagrama de Transição de Estado



2. Término do Processo

O processo encerra voluntariamente sua execução e **libera a CPU**.

Exemplos Práticos:

- Finalização normal da instrução final / ***return***
- Invocação explícita da chamada ***exit()***

Diagrama de Finalização



Scheduling sem Preempção

Características e funcionamento do modelo cooperativo de escalonamento

Regras Principais

- Nas situações de liberação, **não há alternativa** a não ser selecionar um novo processo na fila de prontos.
- O processo **não é interrompido** pelo Sistema Operacional.
- A CPU só é liberada quando o próprio processo encerra ou solicita E/S.

Quando o *scheduling* ocorre apenas sob estas circunstâncias, o esquema é chamado de **sem preempção** ou **cooperativo**.

Fluxo de Execução Cooperativo

1. Processo Atual em Execução
Controle total da CPU (SO não intervém)

↓ **Termina ou Solicita E/S**

2. Liberação Voluntária da CPU
O processo encerra ou entra em Espera

↓ **Seleção Obrigatória**

3. Novo Processo Selecionado
Escolhido diretamente da Fila de Prontos

Scheduling com Preempção

Mecanismo de interrupção forçada de processos pelo Sistema Operacional

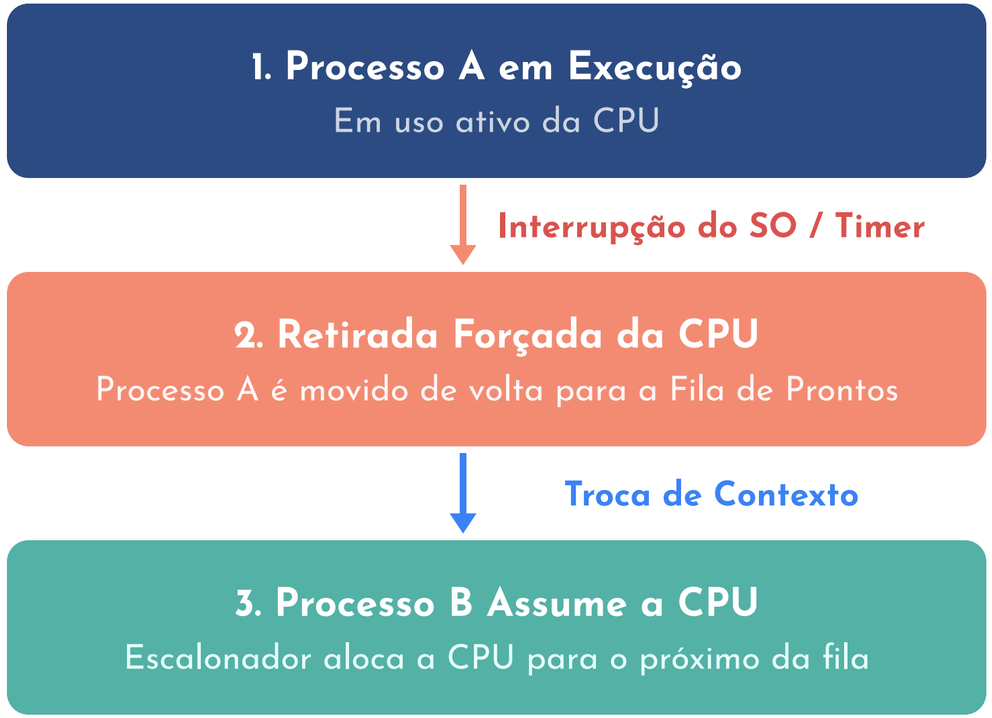
Definição Fundamental

O **Sistema Operacional** pode forçar a saída de um processo da CPU, mesmo que ele não tenha terminado ou solicitado I/O.

- **Interrupção Forçada: O SO retoma o controle a qualquer momento.**
- **Temporizador (Timer): O hardware gera interrupções periódicas.**
- **Multitarefa Justa: Impede que um processo monopolize o processador.**

Exemplo Prático: Quando o tempo do processo atual expira (quantum de tempo), o SO salva seu estado na fila de **Pronto** e concede a CPU a outro processo.

Diagrama de Preempção



Scheduling com Preempção

O risco de Condições de Corrida (Race Conditions) em dados compartilhados

Condição de Corrida

Ocorre quando o **scheduling com preempção** interrompe a execução de processos que compartilham a mesma área de memória.

- **Dados Compartilhados: Dois ou mais processos acessam os mesmos dados.**
- **Interrupção Precoce: Um processo é interrompido antes de concluir a gravação.**
- **Inconsistência: O próximo processo lê informações parciais ou corrompidas.**

Consequência Direta: A leitura de dados em estado inconsistente pode causar falhas graves e comportamento imprevisível no sistema.

Exemplo de Concorrência

1. Processo 1 Inicia Atualização

Modifica apenas metade dos dados compartilhados

↓ **Interrupção (Preempção)**

2. Processo 2 Assume a CPU

Tenta ler os dados enquanto o Processo 1 está pausado

↓ **Leitura Concorrente**

3. Estado Inconsistente Gerado

O Processo 2 obtém informações parcialmente desatualizadas

Scheduling com Preempção

Proteção e Sincronização em Seções Críticas do Kernel

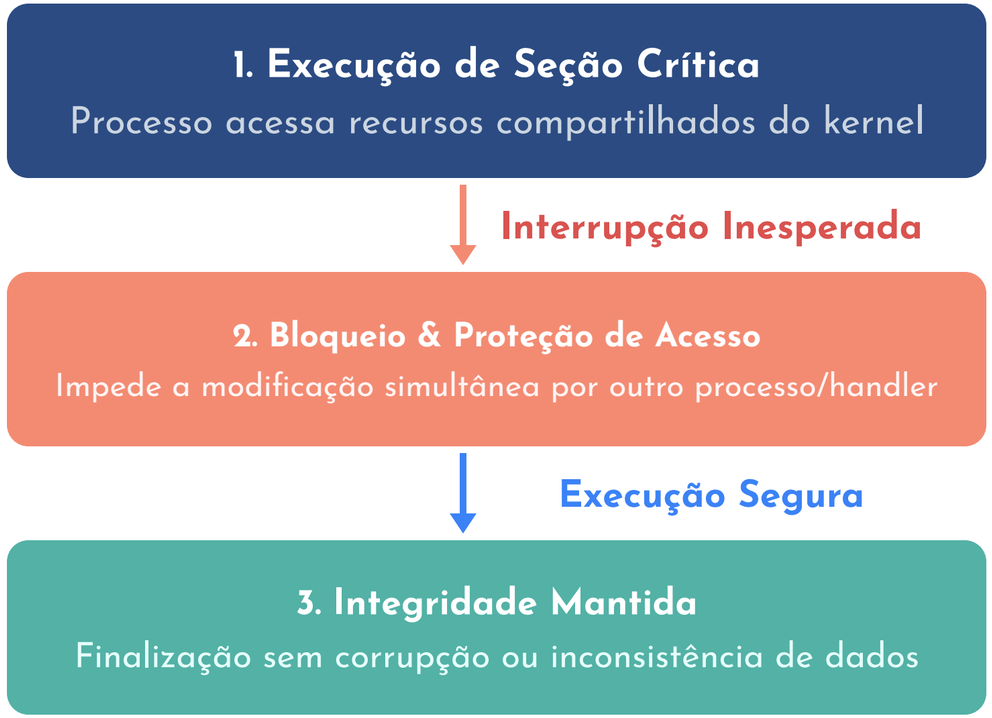
Necessidade de Proteção

Como as interrupções ocorrem a qualquer instante, o código do kernel precisa de controle rigoroso de acesso.

- **Interrupções Constantes: Podem ocorrer a qualquer momento e não podem ser ignoradas.**
- **Seções Sensíveis: Trechos de código afetados devem ser protegidos contra uso simultâneo.**
- **Consistência do Kernel: Evita a corrupção de dados e falhas do sistema.**

Requisito Vital do SO: O sistema operacional precisa aceitar interrupções a quase todo momento para manter a reatividade.

Fluxo de Proteção de Código



Despachante (Dispatcher)

Módulo de passagem de controle da CPU para o processo selecionado

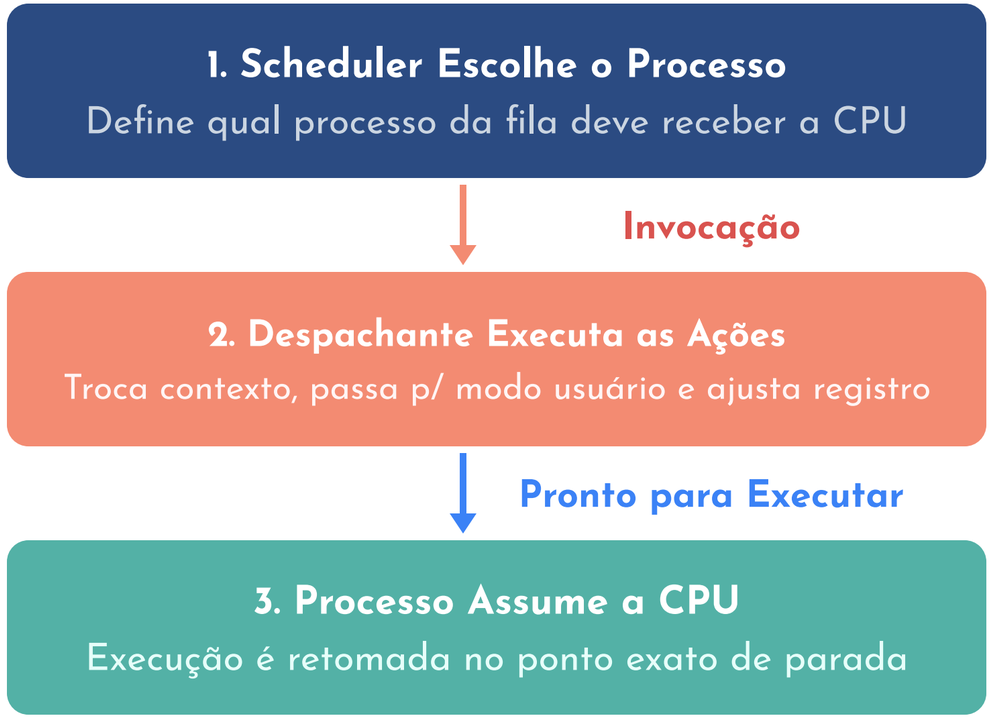
Funções Principais

O **despachante** é invocado pelo **scheduler de curto prazo** para realizar 3 etapas essenciais:

- **Mudança de Contexto:** Salva o estado do processo antigo e carrega o do novo.
- **Modo Usuário:** Alterna o modo da CPU de kernel para usuário.
- **Salto de Execução:** Desvia para a instrução correta no programa para reiniciá-lo.

Papel Fundamental: Garantir a transição suave e imediata entre tarefas para manter o sistema responsivo.

Fluxo do Despachante



Parte II - Critérios de Scheduling

Critérios de Scheduling

Seleção e Comparação de Algoritmos de CPU

1. Diversidade de Algoritmos

Diferentes algoritmos de scheduling da CPU têm **propriedades distintas**.

A escolha de um algoritmo específico pode **favorecer uma classe de processos** em vez de outra.

Exemplo: Impacto no Perfil do Processo

Processos I/O-Bound

Esteriótipo: Uso de CPU curto e frequente

Processos CPU-Bound

Esteriótipo: Cálculo intensional prolongado

2. Comparação e Avaliação

Muitos critérios têm sido sugeridos para a **comparação de algoritmos** de scheduling da CPU.

As características usadas na comparação fazem uma **grande diferença** na avaliação do algoritmo ideal.

Objetivos Principais de Avaliação

- **Eficiência:** Maximizar o uso da CPU
- **Produtividade:** Maximizar o Throughput
- **Tempo de Resposta:** Minimizar a espera

Critérios de Scheduling: Throughput

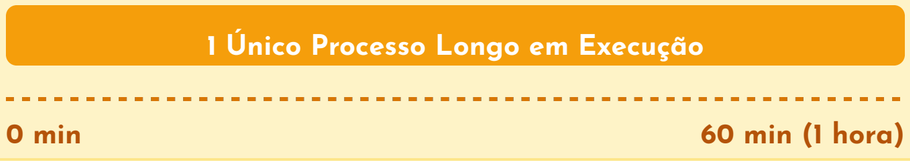
Mede a quantidade de trabalho realizada pela CPU em um intervalo de tempo.

O que é Throughput? É o **número de processos concluídos por unidade de tempo**. Se a CPU está executando processos ativamente, o trabalho útil está sendo realizado.

Exemplo Prático: Variação da Taxa de Throughput

Processos Longos

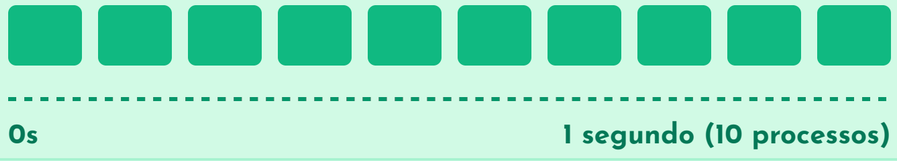
Ex: Tarefas complexas de computação, renderização ou relatórios heavy-duty.



Taxa de Throughput: 1 processo / hora

Transações Curtas

Ex: Requisições web, consultas de banco de dados ou cliques do usuário.



Taxa de Throughput: 10 processos / segundo

Critérios de Scheduling: Tempo de Turnaround

Mede o intervalo total decorrido entre a submissão de um processo e a sua conclusão.

O que é Tempo de Turnaround?

Do ponto de vista de um processo específico, é o tempo total necessário para sua execução completa. Quanto menor este tempo, melhor o desempenho percebido.

Composição do Tempo de Turnaround (Soma dos Períodos)

1. Espera na Memória

Tempo gasto aguardando a alocação de memória principal para que o processo possa ser carregado.

2. Fila de Prontos

Tempo despendido na fila de processos prontos, esperando ser selecionado pela CPU.

3. Execução na CPU

Tempo efetivo em que o processo está sendo processado ativamente pelo processador.

4. Operações de I/O

Tempo gasto aguardando ou realizando operações de Entrada/Saída (disco, rede, etc.).

Fórmula Resumida:

Tempo de Turnaround = **Espera Memória** + **Espera Fila Prontos** + **Execução CPU** + **Execução I/O**

Critérios de Scheduling: Tempo de Espera

Mede o tempo total despendido por um processo aguardando na fila de prontos.

O que é Tempo de Espera?

É a **soma dos períodos** gastos em espera na fila de prontos. O algoritmo afeta **somente** o tempo na fila, sem alterar a execução ou o I/O.

Entendendo o Impacto do Escalonador

O Algoritmo NÃO Afeta

Factores independentes da estratégia de scheduling:

1. Execução na CPU: Duração efetiva do processamento do código.

2. Operações de I/O: Tempo de leitura/escrita em disco e rede.

Exemplo: Se um processo precisa de 5s de CPU e 3s de I/O, esses tempos não mudam.

O Algoritmo AFETA Direta e Exclusivamente

O principal indicador de eficiência da fila:

- **Permanência na Fila de Prontos:** Tempo parado aguardando a CPU ser liberada.
- **Redução de Gargalos:** Bons algoritmos minimizam a espera dos processos.

Fórmula Simplificada:

Tempo de Espera Total = ∑ (Espera 1 + Espera 2 + ...)

Critérios de Scheduling: Tempo de Resposta

Mede o tempo da submissão de uma solicitação até a produção da primeira resposta.

O que é Tempo de Resposta?

Em um sistema interativo, é o tempo decorrido desde a **submissão da solicitação** até a exibição da **primeira saída/resposta** ao usuário.

Por que é Crucial em Sistemas Interativos?

1. Percepção do Usuário: O tempo de turnaround total pode ser longo, mas a resposta rápida dá sensação de agilidade.

2. Saída Incremental: O processo gera novos resultados em segundo plano enquanto exibe os anteriores.

Exemplo Prático:

Ao pesquisar em uma loja online, os primeiros produtos surgem na tela em **poucos milissegundos** (Tempo de Resposta), embora a página continue carregando outros dados ao fundo.

Linha do Tempo de Execução



Diferença-Chave:

O **Tempo de Resposta** mede o início do retorno, não a conclusão total.

Critérios de Scheduling: Metas e Otimização

Como o sistema operacional decide balancear os recursos para os processos.

O que buscamos alcançar no Escalonamento?

▲ **MAXIMIZAR:** Utilização da CPU e Throughput (vazão de processos)

▼ **MINIMIZAR:** Tempo de Turnaround, Tempo de Espera e Tempo de Resposta

1. Abordagem Padrão

Otimização da Medida Média

- **Foco Geral:** Na maioria dos casos, os algoritmos buscam otimizar o desempenho médio do sistema.
- **Objetivo:** Garantir que o rendimento global seja eficiente para a maioria dos processos.

Figura: Equilíbrio Médio

[Processo A: 2s] + [Processo B: 4s] + [Processo C: 3s]

Tempo Médio de Espera = (2 + 4 + 3) / 3 = 3s

2. Casos Especiais

Otimização de Mínimos e Máximos

- **Garantia de Qualidade:** Sob algumas circunstâncias, é preferível limitar os extremos em vez da média.
- **Equidade:** Evita que algum usuário sofra com atrasos inaceitáveis no pior caso.

Exemplo Prático:

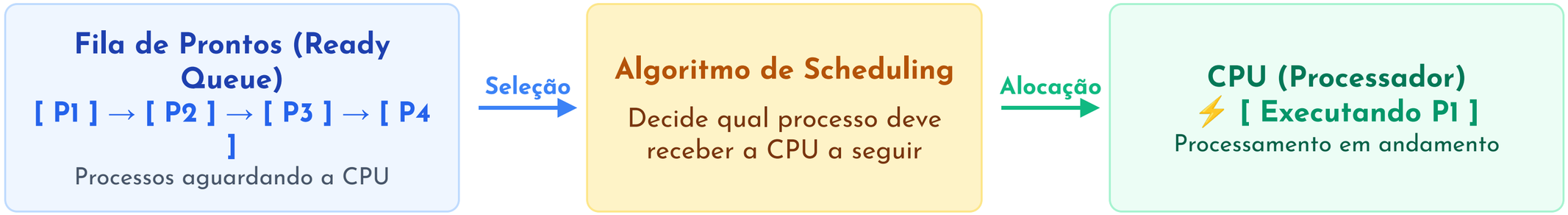
Para garantir um serviço satisfatório para **TODOS** os usuários, minimiza-se o **Tempo de Resposta Máximo** (ninguém espera muito).

Parte III - Algoritmos de Scheduling

Algoritmos de Scheduling

Gerenciamento e Alocação do Processador

Figura 1: Funcionamento Geral do Escalonamento da CPU



O Problema Central

Tomada de decisão em tempo real

- O **scheduling da CPU** lida com a decisão de qual processo da fila de prontos receberá o processador.
- **Objetivo:** Alternar a CPU entre processos para tornar o computador mais produtivo e responsivo.

Diversidade de Algoritmos

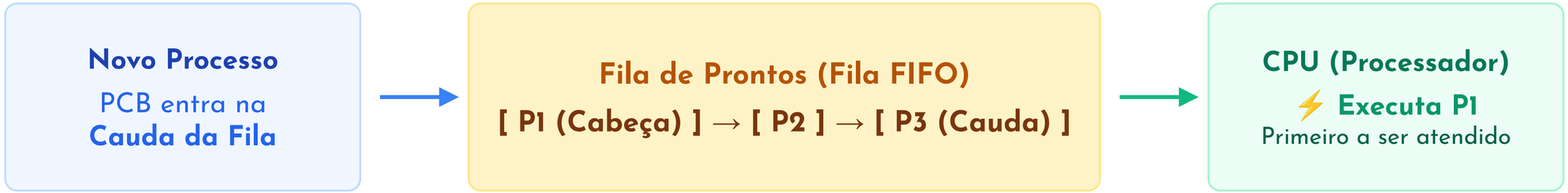
Diferentes estratégias de otimização

- Há **muitos algoritmos diferentes**, cada um com critérios e metas específicas.
- **Exemplos que veremos:** FCFS (Primeiro a chegar), SJF (Menor tempo primeiro), Round Robin e Prioridades.

"Primeiro a chegar, primeiro a ser atendido" (FCFS)

Algoritmo de Escalonamento da CPU Baseado em Fila FIFO

Figura: Fluxo do Escalonamento FCFS / FIFO



O Algoritmo FCFS

First-Come, First-Served

- É o algoritmo de *scheduling* da CPU **mais simples** que existe.
- O processo que solicita a CPU primeiro é o **primeiro a usá-la**.

Funcionamento em Fila (FIFO)

First-In, First-Out

- Segue rigorosamente a estrutura de uma **fila tradicional (FIFO)**.
- Quando um processo entra na fila de prontos, seu **PCB é conectado na cauda da fila**.

"Primeiro a chegar, primeiro a ser atendido" (FCFS)

Exemplo do Algoritmo de Escalonamento da CPU

Considere o conjunto de processos a seguir que chegam no tempo 0, com o intervalo do pico de CPU dado em milissegundos:

Tabela de Processos

Processo	Duração do Pico
P_1	24 ms
P_2	3 ms
P_3	3 ms

Tempo Total de Processamento = 30 ms

Chegada no Tempo t = 0

Fila de Prontos (FIFO)

[P_1] → [P_2] → [P_3]

(Cauda) (Cabeça)

→ **CPU** ⚡ P_1

Regras do Exemplo:

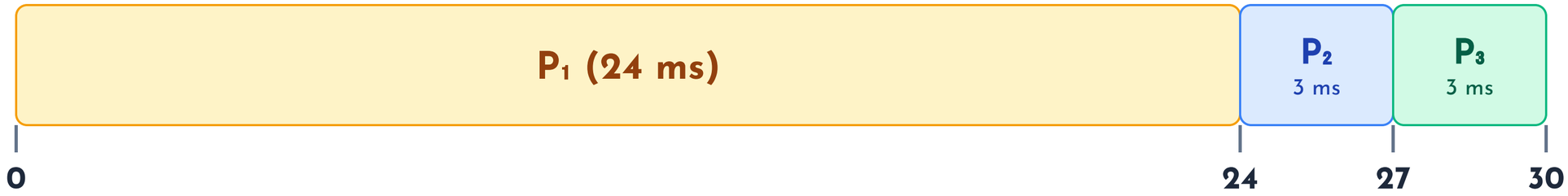
- Todos os três processos (P_1, P_2, P_3) chegam **simultaneamente no tempo 0**.
- **P_1** é atendido primeiro por ocupar a cabeça da fila.
- **P_2** e **P_3** aguardam em ordem na fila até a liberação da CPU.

"Primeiro a chegar, primeiro a ser atendido" (FCFS)

Exemplo 1: Processos na ordem de chegada $P_1 \rightarrow P_2 \rightarrow P_3$

Se os processos chegam na ordem **P_1**, **P_2** e **P_3** e são atendidos via **FCFS**, a CPU os executa sequencialmente:

Gráfico de Gantt (Linha do Tempo de Execução)



Tempos de Espera: Processo **P_1** = **0 ms** Processo **P_2** = **24 ms** Processo **P_3** = **27 ms**

Cálculo do Tempo Médio de Espera

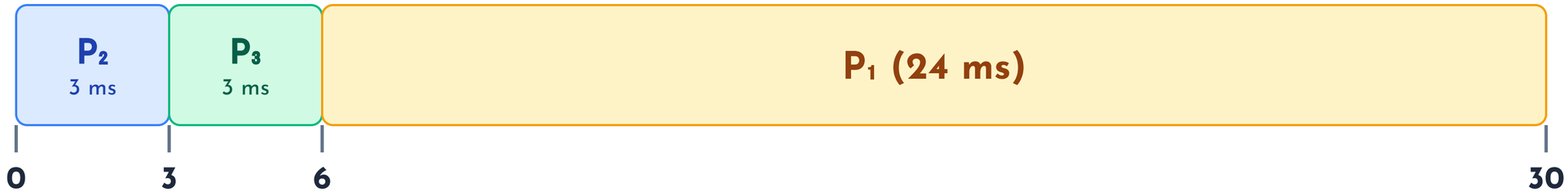
(0 + 24 + 27) / 3 = **17 ms**

"Primeiro a chegar, primeiro a ser atendido" (FCFS)

Exemplo 2: Processos na ordem de chegada $P_2 \rightarrow P_3 \rightarrow P_1$

Se os processos chegam na ordem **P_2**, **P_3** e **P_1**, a ordem de execução no Gantt altera drasticamente os tempos:

Gráfico de Gantt (Linha do Tempo de Execução)



Tempos de Espera: Processo **P_2 = 0 ms** | Processo **P_3 = 3 ms** | Processo **P_1 = 6 ms**

Cálculo do Tempo Médio de Espera

$$(0 + 3 + 6) / 3 = \mathbf{3\ ms}$$

⚡ **Essa redução de 17 ms para 3 ms é substancial!**

"Primeiro a chegar, primeiro a ser atendido" (FCFS)

Análise de Desempenho e Limitações do Algoritmo

🚫 **Algoritmo SEM PREEMPÇÃO:** Uma vez alocada a CPU a um processo, ele a mantém até concluir sua execução ou solicitar E/S.

⏱️ Tempo de Espera

Não é mínimo / ótimo

O tempo médio varia drasticamente dependendo da ordem de chegada.

Se processos com longos picos de CPU chegarem primeiro, todos os processos subsequentes sofrerão altos tempos de espera.

🚛 Efeito Comboio

Ocorre quando **vários processos curtos** ficam retidos aguardando um **processo longo** liberar a CPU.

Ilustração: Fila de Execução FCFS Processo P_1 Muito Longo (CPU-bound) ⚡ EM EXECUÇÃO P_2 Curto P_3 Curto ⏳ AGUARDANDO

Consequência: Baixa utilização de dispositivos de I/O e queda no desempenho geral do sistema.

📌 Em Resumo

Simplicidade

Fácil de implementar usando uma fila FIFO simples.

Justiça Simples

Atende rigorosamente pela ordem de chegada.

Ineficiência

Inadequado para sistemas interativos ou de tempo compartilhado.

"Menor job primeiro" (SJF)

Conceito e Funcionamento do Algoritmo Shortest-Job-First

💡 O algoritmo SJF associa o tempo de execução ao próximo pico de CPU de cada processo.

🎯 Associação do Pico

Métrica do Processo

O algoritmo de scheduling menor-job-primeiro (SJF – *shortest-job-first*) associa a cada processo a duração do próximo pico de CPU do processo.

⚡ Atribuição da CPU

Prioridade ao Menor Tempo

Quando a CPU está disponível, ela é atribuída ao processo que tem o próximo pico de CPU mais curto.

⚖️ Desempate

Regra FCFS em Empates

Se os próximos picos de CPU de dois processos forem iguais, o scheduling FCFS será usado para resolver o impasse.

"Menor job primeiro" (SJF)

Exemplo Prático: Conjunto de Processos e Pico de CPU

📊 Considere o seguinte conjunto de processos com suas durações de pico de CPU em milissegundos:

Processo P_1

Duração do Pico

6 ms

Carga de CPU:

Processo P_2

Duração do Pico

8 ms

Carga de CPU:

Processo P_3

Duração do Pico

7 ms

Carga de CPU:

Processo P_4

Duração do Pico

3 ms

⚡ **Menor Pico!**

Carga de CPU:

💡 **Ordem de Execução no Algoritmo SJF:**

O escalonador seleciona primeiro o processo com menor tempo de execução: **P_4 (3ms) (cabeça) → P_1 (6ms) → P_3 (7ms) → P_2 (8ms)**

"Menor job primeiro" (SJF)

Gráfico de Gantt e Cálculo do Tempo Médio de Espera

📊 Usando o scheduling SJF, os processos são executados por ordem do menor pico de CPU: P_4 (3ms) ➔ P_1 (6ms) ➔ P_3 (7ms) ➔ P_2 (8ms)

Processo P_1

Duração do Pico

6 ms

Processo P_2

Duração do Pico

8 ms

Processo P_3

Duração do Pico

7 ms

Processo P_4

Duração do Pico

3 ms

📈 Gráfico de Gantt (Escalonamento SJF)



⚡ Tempo Médio de Espera (SJF)

Tempo de espera de cada processo:
P_4 = 0ms | P_1 = 3ms | P_3 = 9ms | P_2 = 16ms

Cálculo: (3 + 16 + 9 + 0) / 4

7,0 ms

🐢 Comparativo: Tempo Médio de Espera (FCFS)

Em ordem de chegada tradicional (FCFS):
P_1 = 0ms | P_2 = 6ms | P_3 = 14ms | P_4 = 21ms

Cálculo: (0 + 6 + 14 + 21) / 4

10,25 ms

"Menor job primeiro" (SJF)

Vantagem Principal e o Desafio da Estimativa do Tempo de CPU

✨ Desempenho Ótimo

O algoritmo de scheduling SJF é comprovadamente **ótimo**, pois fornece o **tempo médio de espera mínimo** para qualquer conjunto de processos.

⚠️ A Grande Dificuldade: Como saber a duração da próxima solicitação de CPU?

É necessário estimar o limite de tempo do processo de maneira precisa antes da execução.

📊 Impacto da Precisão na Estimativa de Tempo:

1. Estimativa Alta / Conservadora

Valor de Tempo Elevado

- Pode garantir que o job termine sem ser interrompido.
- **Porém:** O tempo de resposta esperado pode parecer mais lento ou perder prioridade na fila SJF.

2. Estimativa Baixa Demais ⚠️

💥 Erro: Limite Excedido (Timeout)

- Um valor mais baixo promete resposta rápida na teoria...
- **Consequência Grave: O processo estoura o tempo limite estipulado.**
- **Exige uma nova submissão completa do job!**

"Menor job primeiro" (SJF)

Comportamento com Preempção vs. Sem Preempção

⚡ Momento da Decisão no Escalonamento

A escolha é feita quando um novo processo chega à fila de prontos enquanto um processo anterior ainda está sendo executado.

1. SJF Com Preempção

🔄 Interrompe a Execução Corrente

- O novo processo tem pico de CPU **mais curto** do que o tempo remanescente do atual.
- O algoritmo **interrompe imediatamente** o processo em execução.
- O processo interrompido retorna à fila de prontos.

2. SJF Sem Preempção

⏳ Conclui o Pico Atual

- Mesmo que o novo processo seja mais curto, não há interrupção imediata.
- **Permitirá que o processo corrente termine** completamente seu pico de CPU.
- O novo processo aguarda na fila de prontos sua vez de ser executado.

💡 Regra Comparativa Essencial:

A comparação não é feita apenas com o tempo total do processo, mas com o **tempo remanescente** do processo em execução. Se o novo pico de CPU for menor, o escalonador com preempção troca o processo imediatamente.

"Menor job primeiro" (SJF)

Exemplo Prático: Tabela de Chegada e Duração dos Processos

Processo	Tempo de Chegada	Duração do Pico (CPU)
P_1	0 ms	**8 ms**
P_2 ★	1 ms	**4 ms (Menor)**
P_3	2 ms	**9 ms**
P_4	3 ms	**5 ms**

⚡ Análise dos Dados

- **Ordem de Chegada:** Os processos entram na fila entre 0 ms e 3 ms.
- **Tempo de Pico:** Indica a duração de execução exigida na CPU.

💡 Escolha do SJF (P_2):

Entre os processos disponíveis, P_2 apresenta a menor duração (4 ms), recebendo prioridade na fila de execução.

📌 Conceito Fundamental do Escalonamento SJF

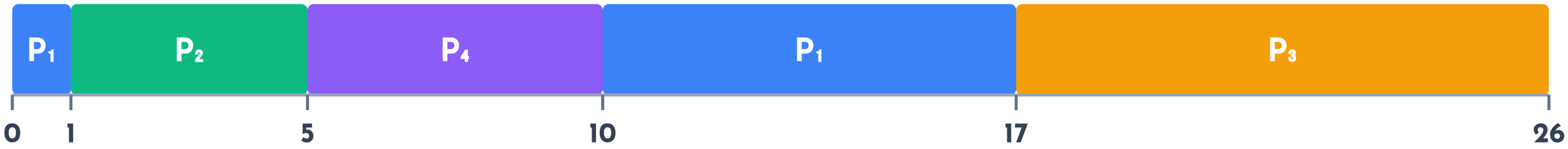
O algoritmo prioriza o processo com o **menor tempo de CPU**, minimizando o tempo médio de espera no sistema.

"Menor job primeiro" (SJF)

Exemplo Prático: Gráfico de Gantt e Tempo Médio de Espera

⚡ Se os processos chegarem nos momentos indicados, o scheduling SJF com preempção resulta no seguinte gráfico de Gantt:

📊 Execução dos Processos no Tempo (Gráfico de Gantt)



🧮 Tempo de Espera (Com Preempção)

- **P_1:** (10 – 1) = 9 ms
- **P_2:** (1 – 1) = 0 ms
- **P_3:** (17 – 2) = 15 ms
- **P_4:** (5 – 3) = 2 ms

T médio = (9 + 0 + 15 + 2) / 4 = 26 / 4 = 6,5 ms

⏳ Sem Preempção

Comparativo de desempenho:

T médio (sem preempção) = 7,5 ms

💡 A preempção otimiza o fluxo e reduz o tempo médio em **1,0 ms**.

Escalonamento por "Prioridades"

Conceitos Fundamentais e Regras do Algoritmo

🎯 Conceito Geral

O algoritmo **SJF (Shortest Job First)** é um **caso especial** do algoritmo geral de scheduling por prioridades.

🔝 Regra de Alocação da CPU

Uma **prioridade** é associada a cada processo. A CPU é alocada ao processo com a **prioridade mais alta**.

⚖️ Empate de Prioridades

Processos com **prioridades iguais** são organizados no schedule estritamente em ordem **FCFS (First-Come, First-Served)**.

💡 Relação com o SJF

No SJF, a prioridade **(p)** é o **inverso do próximo pico de CPU previsto** (menor tempo = maior prioridade).



"Prioridades"

Por exemplo:

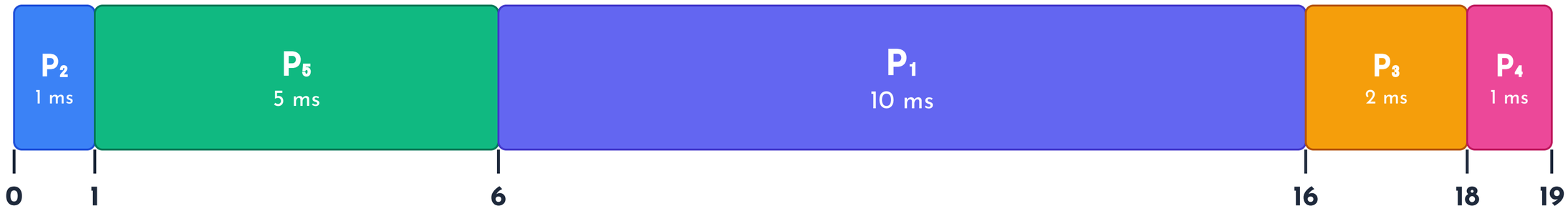
Processo	Duração do Pico	Prioridade
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Escalonamento por "Prioridades"

Organização do Schedule e Cálculo do Tempo Médio de Espera

Usando o scheduling por prioridades, os processos são organizados no schedule de acordo com o gráfico de Gantt a seguir:

📊 Gráfico de Gantt (Linha do Tempo de Execução)



⏱️ Tempo Médio de Espera

8,2 ms

🧮 Detalhamento do Tempo de Espera por Processo:

• P_2: 0 ms | • P_5: 1 ms | • P_1: 6 ms | • P_3: 16 ms | • P_4: 18 ms

Média = (0 + 1 + 6 + 16 + 18) / 5 = 41 / 5 = **8,2 milissegundos**

Escalonamento por "Prioridades"

Como as prioridades são calculadas e atribuídas aos processos

💡 As prioridades dos processos no Sistema Operacional podem ser definidas interna ou externamente:

⚙️ Prioridades Internas

Usam quantidades mensuráveis do próprio processo para calcular sua prioridade de execução.

📊 Métricas e Métodos Utilizados:

- **Limites de Tempo:** Tempo de CPU máximo permitido
- **Requisitos de Memória:** Quantidade de RAM alocada
- **Arquivos Abertos:** Número de arquivos ativos
- **Perfil de E/S:** Razão entre taxa de E/S e uso de CPU

🌐 Prioridades Externas

Definidas por critérios externos ao Sistema Operacional, baseadas em políticas de uso.

🏢 Fatores e Critérios Externos:

- **Importância do Processo:** Criticidade para o negócio
- **Fundos e Pagamentos:** Montante pago pelo uso do computador
- **Departamento Solicitante:** Hierarquia e privilégios
- **Políticas de Uso:** Diretrizes administrativas e operacionais

Escalonamento por "Prioridades"

Mecanismos de Preempção, O Problema da Inanição e A Solução por Envelhecimento

⚡ Com Preempção (Preemptivo)

Se a prioridade do processo recém-chegado for **MAIOR** que a do processo atual:

- **O novo processo interrompe e toma a CPU imediatamente.**
- O processo interrompido retorna para a fila de prontos.

🛑 Sem Preempção (Não-Preemptivo)

Se a prioridade do processo recém-chegado for **MAIOR** que a do processo atual:

- **O novo processo é colocado na cabeça da fila de prontos.**
- O processo em execução continua na CPU até finalizar ou fazer E/S.

⚠️ O Problema: Inanição (Starvation)

Bloqueio Indefinido de Processos

- Em sistemas muito carregados, um fluxo contínuo de processos de alta prioridade impede que processos de baixa prioridade usem a CPU.

📌 Exemplo Histórico:

No MIT (1973), ao desligar o IBM 7094, encontraram um processo submetido em 1967 que **nunca tinha sido executado**!

💡 A Solução: Envelhecimento (Aging)

Aumento Gradual de Prioridade

- Consiste em aumentar gradualmente a prioridade dos processos que permanecem muito tempo aguardando na fila.

📈 Como funciona na prática:

Se a prioridade varia de 127 a 0 (onde 0 é mais alta), aumentar a prioridade em 1 a cada 15 minutos garante que o processo eventualmente **atinja a prioridade máxima** e seja executado!

Escalonamento "Round-Robin" (RR)

Conceitos fundamentais do algoritmo para sistemas de tempo compartilhado

💻 1. Sistemas de Tempo Compartilhado

O algoritmo Round-Robin (RR) foi projetado especialmente para esse ambiente.

- Distribui a CPU entre múltiplos usuários/processos interativos.
- Garante tempo de resposta rápido para todos os programas ativados.

⚡ 2. FCFS com Preempção

Ele é semelhante ao FCFS, mas adiciona a preempção.

- Permite ao sistema interromper o processo em execução.
- Alterna periodicamente o uso do processador entre os processos.

⏳ 3. Quantum de Tempo (Time Slice)

Definição de Fatia de Tempo:

Uma pequena unidade de tempo, chamada **quantum de tempo** ou **porção de tempo**, é definida no sistema.

Cada processo tem direito a usar a CPU por no máximo 1 quantum por vez.

⏱️ 4. Duração do Quantum

Faixa Típica de Operação:

10 a 100 milissegundos

Geralmente um quantum de tempo tem essa duração nos Sistemas Operacionais modernos.

Escalonamento "Round-Robin" (RR)

Cenário 1: Liberação Voluntária da CPU

🟢 Opção 1: Pico de CPU menor que 1 Quantum (< 1 Quantum)

O que acontece: O processo executa e finaliza sua tarefa (ou solicita E/S) **antes** do tempo limite vencer.

- O próprio processo **libera a CPU voluntariamente**.
- O *scheduler* assume imediatamente e passa a CPU para o **próximo processo** na fila de prontos.

📊 Fluxo Visual do Processo



💡 **Exemplo Prático:**

Se o Quantum do sistema for de **10ms** e o Processo P1 precisar de apenas **4ms** de CPU:

P1 executa por 4ms, conclui sua execução e **libera a CPU imediatamente** aos 4ms (sem esperar atingir os 10ms). O escalonador escolhe P2 para usar a CPU logo em seguida.

Escalonamento "Round-Robin" (RR)

Cenário 2: Interrupção por Tempo (Preempção)

🔴 Opção 2: Pico de CPU maior que 1 Quantum (> 1 Quantum)

O que acontece: O processo necessita de mais tempo de CPU do que a fatia limite (quantum) permitida.

- O **timer expira**, é desligado e gera uma **interrupção de hardware** para o Sistema Operacional.
- Uma **mudança de contexto** é executada e o processo é movido para a **cauda da fila de prontos**.
- O *scheduler* da CPU seleciona imediatamente o **próximo processo** da fila para executar.

📊 Fluxo Visual da Interrupção



💡 Exemplo Prático:

Se o Quantum do sistema for de **4ms** e o Processo P1 precisar de **24ms** de CPU:

P1 executa pelos primeiros 4ms. Aos 4ms, o timer é acionado, interrompendo P1. Ele é recolocado no final da fila de prontos (com 20ms restantes) e a CPU é alocada para o Processo P2.

Escalonamento "Round-Robin" (RR)

Análise de Desempenho e Carga de Trabalhos

⚠️ **Observação do Escalonador:**
O tempo médio de espera na política RR é frequentemente longo. Considere o conjunto de processos a seguir que chegam no **tempo = 0ms**:

Processo P_1	Processo P_2	Processo P_3
Duração do Pico de CPU	Duração do Pico de CPU	Duração do Pico de CPU
24 ms	**3** ms	**3** ms

📊 Representação Visual: Fila de Prontos Chegando em t = 0ms

1° na Fila: P_1
Pico de CPU = 24 ms

→

2° na Fila: P_2
Pico de CPU = 3 ms

→

3° na Fila: P_3
Pico de CPU = 3 ms

💡 **Resumo do Cenário:** Ordem de Chegada: **P_1** → **P_2** → **P_3** | Tempo Total de Processamento Requerido: **30 ms**

Escalonamento "Round-Robin" (RR)

Fluxo de Execução Passo a Passo (Quantum = 4ms)

⏱️ **Regra do Sistema: Quantum de Tempo = 4ms**

Acompanhe o comportamento da CPU durante a alternância entre os processos P1, P2 e P3:

1. Início com P1

Tempo: 0ms a 4ms

Se usarmos um quantum de tempo de 4ms, o processo **P1** ficará com os 4 primeiros milissegundos de CPU.

2. Interrupção

Preempção de P1

Já que **P1** requer outros 20ms, ele é interceptado após o primeiro quantum, e a CPU é alocada ao processo **P2**.

3. Execução de P2

Término Antecipado

O processo **P2** não precisa de 4ms e, portanto, é encerrado antes que seu quantum de tempo expire.

4. Próximo Fila (P3)

Liberação da CPU

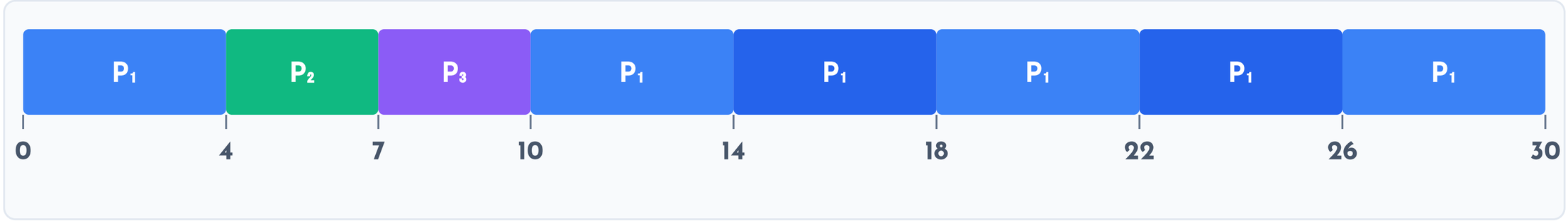
Com o término de P2, a CPU é então liberada para o próximo processo da fila, o processo **P3**.

Escalonamento "Round-Robin" (RR)

Resultado da Distribuição de Tempo da CPU (Diagrama de Gantt)

🔄 **Dinâmica do Escalonamento:**
Após cada processo receber 1 quantum de tempo, a CPU retorna ao processo **P_1** por quanta adicionais até a sua conclusão.

📊 Gráfico de Gantt: Linha do Tempo de Execução da CPU



Legenda: ■ Processo P_1 (24ms) | ■ Processo P_2 (3ms) | ■ Processo P_3 (3ms)

1. Alternância Inicial (0-10ms)

Cada processo (P_1, P_2, P_3) recebe 1 quantum de CPU (até 4ms). P_2 e P_3 concluem nesta fase.

2. Execução Contínua (10-30ms)

Sem outros processos na fila, a CPU é devolvida exclusivamente para P_1 até finalizar o seu tempo restante.

3. Conclusão Total (30ms)

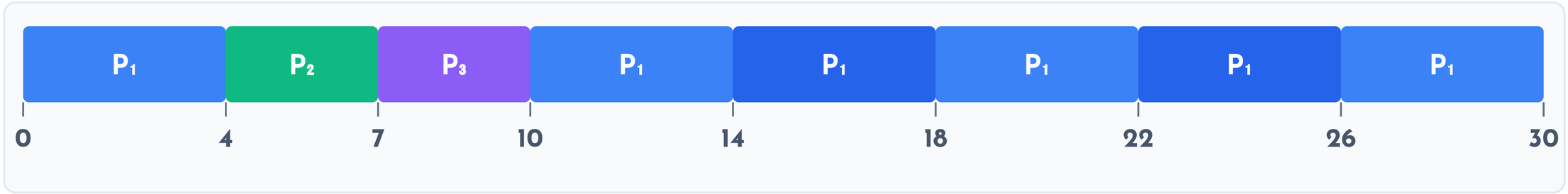
O tempo total decorrido para processar os 3 processos é exatamente **30 ms**.

Escalonamento "Round-Robin" (RR)

Cálculo Prático do Tempo Médio de Espera

🔄 **Dinâmica do Escalonamento:**
Após cada processo receber 1 quantum de tempo, a CPU retorna ao processo P_1 por um quantum adicional.

📊 Diagrama de Execução na CPU (Linha do Tempo em ms)



Legenda: ■ Processo P_1 | ■ Processo P_2 | ■ Processo P_3

1. Espera de P_1

(10 - 4) = 6 ms

P_1 inicia em 0ms, é interrompido em 4ms e só retorna à CPU em 10ms.

2. Espera de P_2 e P_3

P_2 = 4 ms | P_3 = 7 ms

P_2 aguarda de 0ms a 4ms. P_3 aguarda na fila de 0ms até sua vez em 7ms.

3. Tempo Médio Resultante

((10-4) + 4 + 7) / 3

= 5,66 ms

Média do tempo que os 3 processos aguardaram na fila.

Escalonamento "Round-Robin" (RR)

Conceito Fundamental e o Impacto do Tamanho do Quantum

🔄 **Regra do Algoritmo:**
Nenhum processo é alocado à CPU por mais de **1 quantum de tempo** sucessivamente. O desempenho depende diretamente do tamanho desse quantum.

1. Quantum Muito Longo

Efeito: Comportamento FCFS

Se o quantum for maior que o tempo total de execução dos processos, a política RR torna-se idêntica ao **FCFS (First-Come, First-Served)**.

Consequência:

- Sem preempção efetiva.
- Processos curtos aguardam processos longos.
- Sem overhead de troca de contexto.

2. Quantum Equilibrado

Efeito: Escalonamento Ideal

O tempo é dimensionado de forma justa para garantir alternância contínua entre todas as tarefas ativas.

Consequência:

- Boa interatividade no sistema.
- Divisão justa do tempo de CPU.
- Overhead mantido sob controle.

3. Quantum Muito Curto

Efeito: Alto Overhead

Se o quantum for extremamente pequeno (ex: 1ms), ocorrem inúmeras **mudanças de contexto** por segundo.

Consequência:

- Desperdício elevado de ciclos da CPU.
- Lentidão na execução dos processos.
- Degradação geral do desempenho.

Filas Multiníveis (Multilevel Queue Scheduling)

Organização e Particionamento da Fila de Prontos

💡 **Conceito Chave:**
Desenvolvido para atender aplicações com requisitos de tempo de resposta e necessidades de escalonamento completamente distintos.

1. Grupos de Aplicações

Diferenciação de Cargas

Os processos em um sistema não são idênticos. Eles possuem características e expectativas distintas:

- Interativos (ex: Interface gráfica, resposta rápida imediata)
- Segundo Plano (ex: Processamento em lote / Batch)
- Sistema (ex: Serviços vitais do SO)

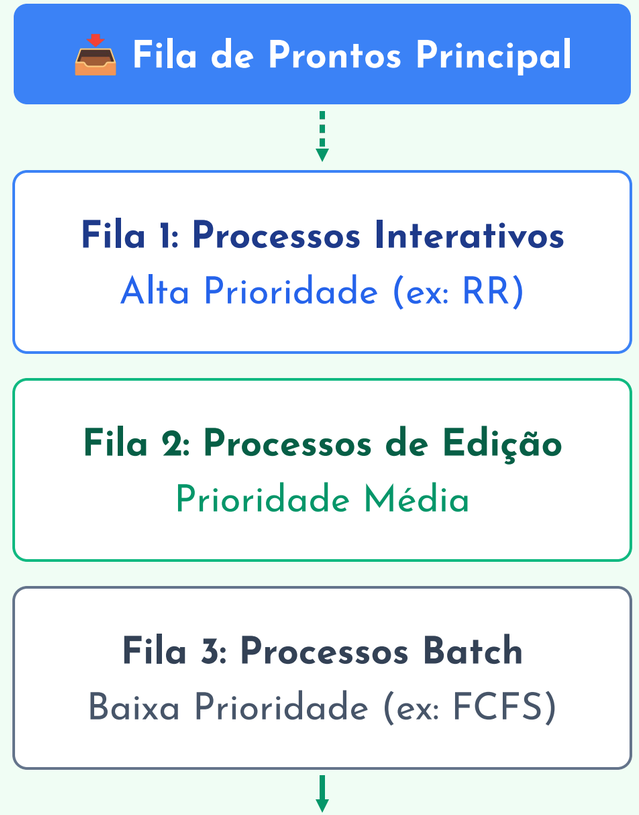
2. Divisão da Fila de Prontos

Particionamento Físico / Lógico

A fila única de prontos é dividida em várias filas independentes e separadas:

- Cada processo é atribuído permanentemente a uma fila.
- A classificação é feita com base em propriedades do processo (memória, prioridade, tipo).
- Evita mistura desordenada de tarefas na CPU.

3. Arquitetura Visual



Filas Multiníveis: Algoritmos de Escalonamento

Funcionamento Independente e Escalonamento Inter-Filas

💡 **Conceito Chave:**
Cada fila possui seu próprio algoritmo de escalonamento, ajustado às necessidades específicas de cada tipo de processo.

1. Divisão de Processos

Tipos de Carga de Trabalho

Filas separadas são utilizadas para categorizar processos com exigências distintas:

- **Foreground:** Processos interativos que exigem resposta rápida.
- **Background:** Processos em lote (batch) executados em segundo plano.

2. Algoritmo por Fila

Estratégia Individual

Cada fila aplica a política mais eficiente para seu tipo de tarefa:

- **Fila Foreground:** Organizada via **Round-Robin (RR)** para garantir alta interatividade.
- **Fila Background:** Organizada via **FCFS** (First-Come, First-Served).

3. Escalonamento Inter-Filas

Gerenciamento de Prioridade

Controle de execução entre as próprias filas do sistema:

- **Prioridade Fixa:** A fila de maior prioridade é sempre atendida primeiro.
- **Preempção:** Processos de foreground interrompem o background em execução.

Filas Multiníveis: Escalonamento por Prioridade

Exemplo Prático de Hierarquia e Preempção de Processos

💡 **Conceito Chave (Prioridade Absoluta):** A fila de foreground tem prioridade total sobre a de background. Nenhuma fila inferior é atendida enquanto houver processos em filas superiores.

Regras de Execução

1. Prioridade Estrita

O escalonador sempre seleciona o processo da fila pronta de maior prioridade disponível.

2. Preempção Automática

Se um processo entra em uma fila superior, a execução atual é interrompida imediatamente.

3. Risco de Inanição (Starvation)

Processos nas filas 4 e 5 só executam se as filas 1, 2 e 3 estiverem totalmente vazias.

Hierarquia das Filas (Ordem de Prioridade)

▲ PRIORIDADE MAIS ALTA

1. Processos do Sistema
Alta relevância / Tarefas vitais do SO

2. Processos Interativos
Interface gráfica e resposta rápida ao usuário

3. Processos de Edição Interativa
Editores de texto e aplicações de escritório

4. Processos Batch
Processamento em lote / Segundo plano

5. Processos de Estudantes
Execuções acadêmicas de menor prioridade

▼ PRIORIDADE MAIS BAIXA

Filas Multiníveis: Divisão do Tempo de CPU

Particionamento de Quotas entre Filas e Escalonamento Independente

💡 **Conceito Chave:** O tempo de CPU é dividido em parcelas fixas entre as filas. Cada fila administra sua quota e a distribui entre seus processos.



Fila Foreground (80% da CPU)

Processos Interativos

- **Algoritmo Utilizado:** Round-Robin (RR).
- **Quota de Processamento:** Recebe 80% de todo o tempo da CPU.
- **Distribuição Interna:** O tempo alocado é fatiado em quânturs e distribuído de forma circular entre seus processos.
- **Objetivo:** Garantir alta interatividade e tempos de resposta ágeis ao usuário.

Fila Background (20% da CPU)

Processos em Lote (Batch)

- **Algoritmo Utilizado:** FCFS (First-Come, First-Served).
- **Quota de Processamento:** Recebe os 20% restantes do tempo da CPU.
- **Distribuição Interna:** Atende os processos por ordem de chegada dentro de sua fatia de tempo reservada.
- **Objetivo:** Executar tarefas de segundo plano de forma previsível sem afetar os processos prioritários.

Filas Multiníveis: Alocação Estática de Processos

Funcionamento, Exemplo Prático e Limitações do Modelo Sem Retroalimentação

💡 **Regra Fundamental:** No escalonamento tradicional de filas multiníveis, a atribuição do processo a uma fila é permanente no seu ingresso.

1. Atribuição Permanente

- **Entrada no Sistema**

O processo é alocado diretamente a uma fila específica assim que é criado.

- **Imutabilidade de Fila**

Permanece na mesma fila durante todo o seu ciclo de execução.

- **Ausência de Transição**

Não há mecanismos de troca ou migração dinâmica entre níveis.

2. Exemplo: Foreground vs. Background

- **Filas Separadas**

Processos interativos (foreground) e em lote (background) têm filas distintas.

- **Natureza Preservada**

Como a classificação do processo não muda, ele nunca troca de fila.

- **Rigidez do Algoritmo**

Mesmo alterando seu perfil de CPU, o processo fica preso à fila inicial.

⚡ Vantagem Principal

Baixo Overhead de Scheduling

Complexidade mínima no gerenciamento do SO, já que não monitora mudanças nem recalcula prioridades.

⚠️ Desvantagem Principal

Alta Inflexibilidade!

Incapaz de responder e se adaptar ao comportamento real e dinâmico dos processos.

Filas Multiníveis com Retroalimentação

Alternância Dinâmica entre Filas conforme o Comportamento do Processo

💡 **Conceito Chave:** Os processos não ficam presos a uma única fila; eles transitam dinamicamente dependendo do uso da CPU e do tempo de espera.

1. Rebaixamento (CPU)

- **Uso Excessivo de CPU**

Se um processo consome muito tempo de processamento (estoura o quantum), ele é rebaixado.

- **Prioridade Mais Baixa**

Ele é transferido para uma fila de prioridade inferior.

- **Objetivo**

Favorecer processos curtos e interativos na fila de alta prioridade.

2. Promoção (Envelhecimento)

- **Longa Espera**

Se um processo aguarda demais em uma fila inferior, o sistema intervém.

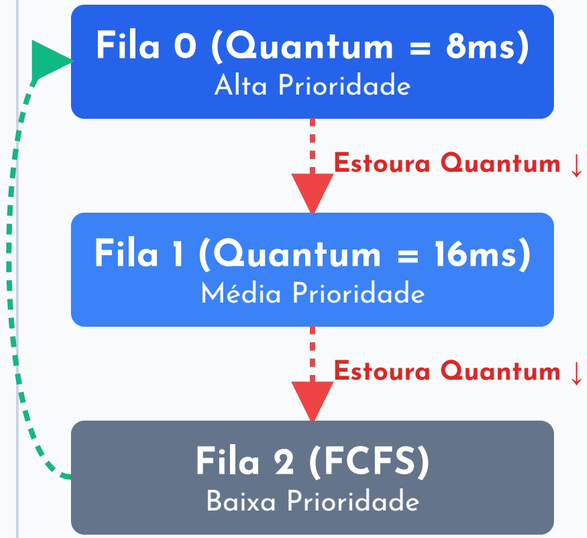
- **Migração para Cima**

O processo é movido para uma fila de maior prioridade.

- **Prevenção de Inanição**

Esse tipo de envelhecimento (Aging) evita totalmente a inanição (Starvation)!

3. Fluxo de Transição



🟢 **Envelhecimento (Aging)**

Processos que esperam muito na Fila 2 sobem de volta para a Fila 0!

Filas Multiníveis com Retroalimentação

Exemplo Prático: Estrutura do Escalonador com 3 Níveis

💡 **Exemplo de Escalonador:** Considere um scheduler de filas multiníveis com retroalimentação manipulando três filas, numeradas de 0 a 2:

Fila 0 (Alta Prioridade)

⚡ **Quantum = 8 ms**

Processos novos entram prioritariamente nesta fila.

- **Escalonamento**

Algoritmo Round Robin (RR) com alta prioridade de CPU.

- **Estouro do Quantum**

Se não terminar em 8ms, o processo é rebaixado para a Fila 1.

Fila 1 (Média Prioridade)

⏱️ **Quantum = 16 ms**

Executa apenas quando a Fila 0 estiver totalmente vazia.

- **Escalonamento**

Algoritmo Round Robin (RR) com tempo de fatia ampliado.

- **Estouro do Quantum**

Se exceder 16ms, o processo é rebaixado para a Fila 2.

Fila 2 (Baixa Prioridade)

📋 **Algoritmo FCFS**

Atendida somente se as Filas 0 e 1 estiverem vazias.

- **Sem Quantum (FCFS)**

Processos mais longos (orientados à CPU) executam até o fim.

- **Mecanismo Anti-Inanição**

O envelhecimento (Aging) pode mover o processo de volta à Fila 0.

Filas Multiníveis com Retroalimentação

Parâmetros Fundamentais de Definição do Escalonador

💡 **Regra de Configuração:** Em geral, um scheduler de filas multiníveis com retroalimentação é definido pelos 5 parâmetros a seguir:

1. Número de Filas

- **Quantidade de Níveis**

Determina o número total de filas de prioridade presentes no sistema.

2. Algoritmo por Fila

- **Algoritmo de Scheduling**

O algoritmo de escalonamento específico aplicado a cada fila individual (ex: RR, FCFS).

3. Elevação (Promoção)

- **Subida de Prioridade**

O método usado para determinar quando um processo deve ser elevado a uma fila de prioridade mais alta.

4. Rebaixamento de Prioridade

- **Descida de Prioridade**

O método usado para determinar quando um processo deve ser rebaixado a uma fila de prioridade mais baixa (ex: ao estourar o fatia de tempo de CPU).

5. Fila de Ingressamento

- **Alocação Inicial**

O método usado para determinar a fila em que um processo entrará quando precisar de serviço pela primeira vez.

Filas Multiníveis com Retroalimentação

Flexibilidade, Otimização e Desafios de Configuração

💡 **Conceito Geral:** Esta é a abordagem mais geral e flexível de escalonamento, porém a mais complexa de parametrizar.

1. Abordagem Geral

- **Visão Abrangente**

O conceito desta abordagem a torna a opção mais geral e abrangente entre os algoritmos.

- **Versatilidade Total**

Permite englobar múltiplos comportamentos de processos em um único modelo.

2. Customização

- **Ajuste Específico**

Pode ser configurado para se ajustar perfeitamente ao sistema específico em projeto.

- **Adaptabilidade**

Permite otimizar os tempos de resposta de acordo com a carga de trabalho desejada.

3. Alta Complexidade

- **Maior Dificuldade**

Infelizmente, também é o algoritmo mais complexo de ser implementado e gerenciado.

- **Seleção de Parâmetros**

A definição do melhor scheduler requer a escolha criteriosa de valores para todos os parâmetros.

Dúvidas?